



**EDUCACIÓN**  
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO  
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO  
DE CHIHUAHUA

## **INSTITUTO TECNOLÓGICO DE CHIHUAHUA**

### **Departamento Eléctrica Electrónica**

### **Material de apoyo teórico para la práctica: Implementación de perceptrón en ESP32 S3**

**Elaborado por:**

**Hector Said Herrera Niño 22061074**

**Adbiel Alejandro Rodríguez Coronado 22061055**

**Profesor: Dr. Juan Alberto Ramírez Quintana**

**Chihuahua, Chih., lunes 29 de junio de 2026**

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Contenido                                                       |    |
| Introducción: .....                                             | 1  |
| Marco teórico.....                                              | 1  |
| ¿Qué es firmware? .....                                         | 1  |
| Modelo de un sistema embebido .....                             | 1  |
| ¿Qué es Hardware Abstraction Layer (HAL)? .....                 | 2  |
| ¿Qué es el Board Support Package (BSP)? .....                   | 2  |
| Punteros en C .....                                             | 2  |
| ¿Qué es un puntero en lenguaje C? .....                         | 2  |
| Operador de dirección &.....                                    | 2  |
| Operador Desreferenciación * .....                              | 3  |
| Arreglos en C .....                                             | 3  |
| ¿Qué es un arreglo? .....                                       | 3  |
| ¿Como declarar un arreglo?.....                                 | 4  |
| ¿Cómo leer el contenido de un elemento de un arreglo? .....     | 4  |
| ¿Cómo escribir el contenido de un elemento de un arreglo? ..... | 4  |
| ¿Cómo inicializar un arreglo? .....                             | 4  |
| Relación de apuntadores y arreglos.....                         | 6  |
| Pasar arreglos como argumento a una función .....               | 6  |
| Arreglos de dos dimensiones .....                               | 8  |
| ¿Como inicializar arreglos de dos dimensiones?.....             | 9  |
| ¿Como iterar dentro de un arreglo de dos dimensiones? .....     | 10 |
| Memoria dinámica.....                                           | 11 |
| Función malloc() .....                                          | 11 |
| Manejando bloques de memoria reservada como arreglos .....      | 11 |
| Función free() .....                                            | 12 |
| Arreglos de longitud variable .....                             | 13 |
| ¿Qué son los arreglos de longitud variable? .....               | 13 |
| VLA como parámetros en funciones .....                          | 14 |
| Memoria dinámica y VLA.....                                     | 16 |
| El perceptrón.....                                              | 16 |
| Elementos básicos del modelo de una neurona .....               | 16 |
| Modelo matemático de una neurona.....                           | 17 |

|                                         |    |
|-----------------------------------------|----|
| Función de activación: Umbral .....     | 17 |
| Algoritmo: LMS (Least-Mean-Square)..... | 18 |
| Bibliografía.....                       | 19 |

## Introducción:

El presente documento tiene el objetivo de brindar los conocimientos básicos para realizar la práctica descrita en el manual de práctica: Implementación de perceptrón en ESP32 S3.

Se abordan 3 principales temas:

- *Software embebido*: donde se aborda el tema de firmware y el modelado en capas de un sistema embebido.
- *Programación en C*: se explican apuntadores, arreglos, VLA y memoria dinámica.
- *Inteligencia artificial*: se habla sobre el modelo de un perceptrón y su entrenamiento por medio del algoritmo LMS.

## Marco teórico

### ¿Qué es firmware?

Zhu (2026) Describe el firmware en un sistema embebido es el programa que está cargado en la memoria no volátil del dispositivo, y es fundamental para el funcionamiento del sistema. En firmware tiene las siguientes funciones:

- Inicialización y arranque.
- Control de Hardware.
- Comunicación entre el hardware y el software.

### Modelo de un sistema embebido

El modelo de un sistema embebido se puede dividir en 3 principales capas (Noergaard, 2005).

- La capa de hardware contiene los componentes físicos que conforman la tablilla embebida.
- La capa de software y aplicación contienen todo el software almacenado y ejecutado por el sistema embebido.

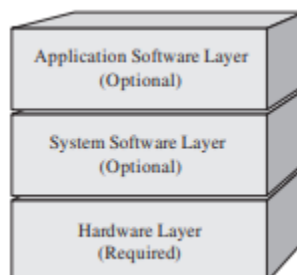


Figura 1. Modelo de sistema embebido. Obtenido de Noergaard (2005).

### ¿Qué es Hardware Abstraction Layer (HAL)?

En esta capa de software se enfoca en ofrecer una API genérica para interactuar con los periféricos de un microcontrolador, además que hace el firmware más portable entre diferentes microcontroladores. (Sami, 2025)

### ¿Qué es el Board Support Package (BSP)?

Combina el HAL con configuraciones específicas de la tablilla donde se desarrolla el sistema. Esto provee interfaces listas para usar para componentes que están sobre la tablilla, como botones, leds, sensores, etc. (impl Rust for ESP32, s.f.)

## Punteros en C

### ¿Qué es un puntero en lenguaje C?

Es una variable que almacena la dirección de la localidad de memoria donde se almacena un dato (Kernighan y Ritchie, 1988).

Cuando un apuntador almacena la dirección de un dato, se dice que ese apuntador “apunta” a esa dirección de memoria.

Al trabajar con apuntadores se necesitará usar el operador de dirección & y el operador de Desreferenciación \*.

La sintaxis para declarar y definir un apuntador es la del listado 1:

|                                                 |
|-------------------------------------------------|
| Tipo_de_dato * nombre_identificador = dirección |
|-------------------------------------------------|

Listado 1. Pseudocódigo que demuestra la sintaxis del listado 1.

Es importante que el tipo de dato del apuntador sea del mismo tipo de dato al que se desea apuntar (Prata, 2004).

### Operador de dirección &

Este operador entrega la dirección de la variable a la cual se le aplique el operador &. El listado 2 muestra cómo se utiliza el operador de dirección cuando se trabaja con apuntadores (Prata, 2004).

|                                                     |
|-----------------------------------------------------|
| int variable1 = 25;<br>int *apuntador = &variable1; |
|-----------------------------------------------------|

Listado 2. Ejemplo en lenguaje C de uso de operador & con apuntadores.

### *Operador Desreferenciación \**

Este operador aparte de ayudar a indicar cuando una variable se comporta como apuntador, así como se usó en el listado 2. También sirve para entregar el valor almacenado en la dirección que apunta el apuntador (Prata, 2004).

El listado 3 muestra la importancia de utilizar el operador de desreferenciación. Tomando en cuenta que apuntador es una variable que almacena una dirección, entonces la variable2 guarda el valor de la dirección, mientras que la variable3 guarda el valor el valor almacenado en la dirección que apunta el apuntador.

```
int variable1 = 25;
int *apuntador = &variable1;

int variable2 = apuntador;
int variable3 = *apuntador;
```

Listado 3. Ejemplo en lenguaje C sobre el uso del operador \* para leer el contenido de la dirección a la que apunta un apuntador.

Nota: Es importante aclarar que a pesar de que la variable2 almacena una dirección, variable2 no funciona como un apuntador, ésta solamente almacena el valor numérico de la dirección.

En la línea de la variable 3 se está leyendo el contenido de la dirección que apunta el apuntador. No obstante, también se puede escribir en el contenido de la dirección que apunta el apuntador.

El listado 4 muestra que la variable1 guarda el valor de 25. Pero luego, por medio del apuntador, se cambia el valor de la variable1 a 44.

```
int variable1 = 25;
int *apuntador = &variable1;

*apuntador = 44;
```

Listado 4. Ejemplo de escritura por medio de apuntadores.

### **Arreglos en C**

#### *¿Qué es un arreglo?*

Es un conjunto de datos ordenados del mismo tipo. Cada elemento que conforma un arreglo tiene asociado un índice, y siempre el primer elemento tiene asociado el índice 0 (Prata, 2004).

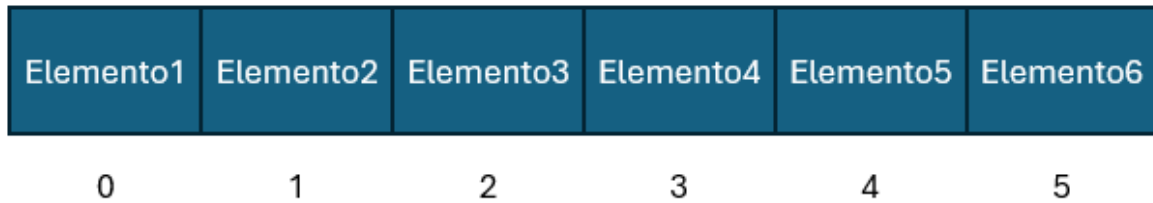


Figura 2. Ejemplo gráfico del concepto de arreglo.

### *¿Como declarar un arreglo?*

La sintaxis para declarar un arreglo es la que se muestra en el listado 5. Es importante este paso, ya que le permite al compilador reservar la cantidad de memoria necesaria (Prata, 2004).

```
Tipo de dato nombre identificador [Cantidad de elementos];
```

Listado 5. Sintaxis para declarar un arreglo.

### *¿Cómo leer el contenido de un elemento de un arreglo?*

El listado 6 se muestra un ejemplo de como asignar el contenido de un elemento de un arreglo a una variable.

```
variable = arreglo[5];
```

Listado 6. Ejemplo de lectura de elemento de un array.

### *¿Cómo escribir el contenido de un elemento de un arreglo?*

El listado 7 muestra cómo asignar un valor a un elemento de un arreglo. Si el elemento ya contenía un valor, este se sobrescribe con el nuevo valor.

```
arreglo[4] = 23;
```

Listado 7. Ejemplo de escritura de elemento de un array.

### *¿Cómo inicializar un arreglo?*

Una vez declarado el arreglo, es necesario inicializarlo, para ello existen 2 principales maneras: usando lista de valores, o utilizando bucle for.

#### *Por medio de una lista de valores.*

El listado 8 muestra un ejemplo de cómo inicializar un array por medio de una lista de valores. Obsérvese que la lista contiene son 9 valores, tal y como se especifica en el tamaño del arreglo (el valor encerrado en []).

```
int array1[9] = {13,15,9,8,2,21,3,41,12};
```

Listado 8. Ejemplo de array inicializado por lista de valores.

Cuando se hace por lista de valores, se puede no indicar la cantidad de elementos que contiene el arreglo. Por ejemplo, el ejemplo anterior también se puede expresar como se muestra en el listado 9 (Deitel y Deitel, 2016).

```
int array1[] = {13,15,9,8,2,21,3,41,12};
```

Listado 9. Ejemplo de array inicializado por lista de valores sin indicar la cantidad de elementos del array.

Tomando en cuenta que el arreglo es de cierto tamaño  $n$ , y la lista de valores de inicialización es de tamaño  $m$ . Si  $n > m$ . Entonces todos los elementos a los que no se les definió un valor quedan como 0. Mencionado lo anterior, entonces en el listado 10, array2 y array3 son equivalentes (Deitel y Deitel, 2016).

```
int array2[9] = {13,15,9,8};  
int array3[9] = {13,15,9,8,0,0,0,0,0};
```

Listado 10. Ejemplo de inicialización en 0 cuando no se define valor para ciertos elementos.

Lo anterior es muy útil cuando se desea inicializar un arreglo con sus valores a 0. Por ejemplo, en el listado 11. Solo se debe definir el primer elemento como 0 en la lista de valores, todos los 99 valores faltantes por inicializar del arreglo tomarán el valor de 0 (Deitel y Deitel, 2016).

```
int array4[100] = {0};
```

Listado 11. Ejemplo de inicializar todo un arreglo con ceros.

*Por medio de bucle for.*

Por medio de la variable de control de un bucle for, es posible iterar a través de los elementos que componen un arreglo, y con ello inicializar cada elemento (Deitel y Deitel, 2016).

Para inicializar el arreglo se puede asignar directamente el valor actual de la variable de control a cada elemento del array como se muestra en el programa del listado 12.

```
int array5[5];  
  
int main()
```

```

{
    for (int i = 0; i<5; i++){
        array5[i] = i;
        printf("%d", array5[i]);
    }
}

```

Listado 12. Ejemplo de inicialización de elementos de un array usando bucle for.

El array5 del listado 12 quedaría visualmente como se describe en la figura 3.

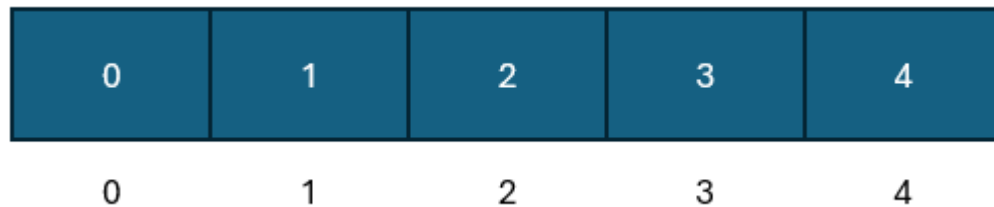


Figura 3. Descripción visual del contenido del array5 del listado 12.

### *Relación de apuntadores y arreglos*

Los apuntadores y arreglos están muy relacionados, ya que, de hecho, la variable con la que se identifica un arreglo realmente es un apuntador al elemento 0 del arreglo. Y por ello toda operación de indexado con arreglos, también se puede realizar con apuntadores (Kernighan y Ritchie, 1988).

Por ello si consideramos un arreglo como el del listado 13.

```

arreglo[5] = {9,5,1,2,4};

```

Listado 13. Ejemplo de arreglo.

La variable “arreglo” es una variable que contiene la dirección del primer elemento del arreglo, por lo tanto, equivale al apuntador del listado 14.

```

apuntador = &arreglo[0];

```

Listado 14. Ejemplo de apuntador que se le asigna el primer elemento de un arreglo.

### *Pasar arreglos como argumento a una función*

Tomando en cuenta el programa del listado 15, donde se muestra una función para imprimir elemento por elemento en una nueva línea. La salida del programa se muestra en el listado 17.

```

#include <stdio.h>

void imprimir_caracteres (char mensaje[]);

char array[] = {'H','o','l','a'};

int main()
{
    imprimir_caracteres(array);
}

void imprimir_caracteres (char mensaje[]){
    for (int i = 0; i < 4; i++){
        printf("%c \n", mensaje[i]);
    }
}

```

Listado 15. Ejemplo de programa donde se envía un arreglo como argumento.

Cuando se pasa un arreglo como un argumento a una función, realmente no se crea una copia local de todo el arreglo, realmente solo se realiza una copia del puntero al primer elemento del arreglo (Kernighan y Ritchie, 1988).

Retomando que la variable identificadora de un arreglo realmente es un puntero al primer elemento del arreglo, el programa del listado 15 equivale al programa del listado 16. Donde en lugar de esperar como argumento un arreglo, se espera un apuntador a char.

```

#include <stdio.h>

void imprimir_caracteres (char *apuntador);

char array[] = {'H','o','l','a'};

int main()
{
    imprimir_caracteres(array);
}

void imprimir_caracteres (char *apuntador){
    for (int i = 0; i < 4; i++){
        printf("%c \n", *(apuntador+i));
    }
}

```

Listado 16. Ejemplo de programa donde se envía un arreglo como argumento usando punteros.

```

H
o
l
a

```

Listado 17. Salida del programa 15 y 16.

### Arreglos de dos dimensiones

Un arreglo de dos dimensiones consiste en un arreglo de una dimensión el cual sus elementos también son arreglos (figura 4) (Kernighan y Ritchie, 1988).

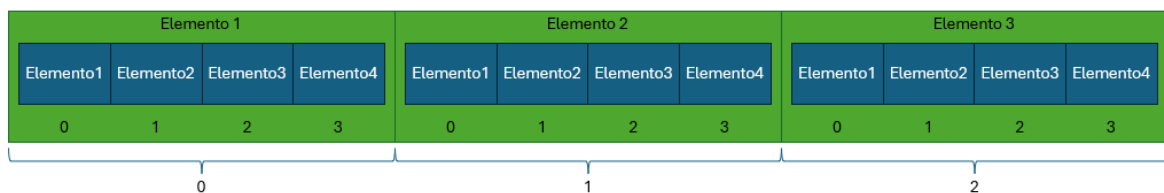


Figura 4. Descripción gráfica de array de dos dimensiones.

Cuando se trabaja con arreglos de dos dimensiones se suele visualizar como una tabla (figura 5), donde cada arreglo representa una fila, y los elementos dentro del arreglo representan las columnas.

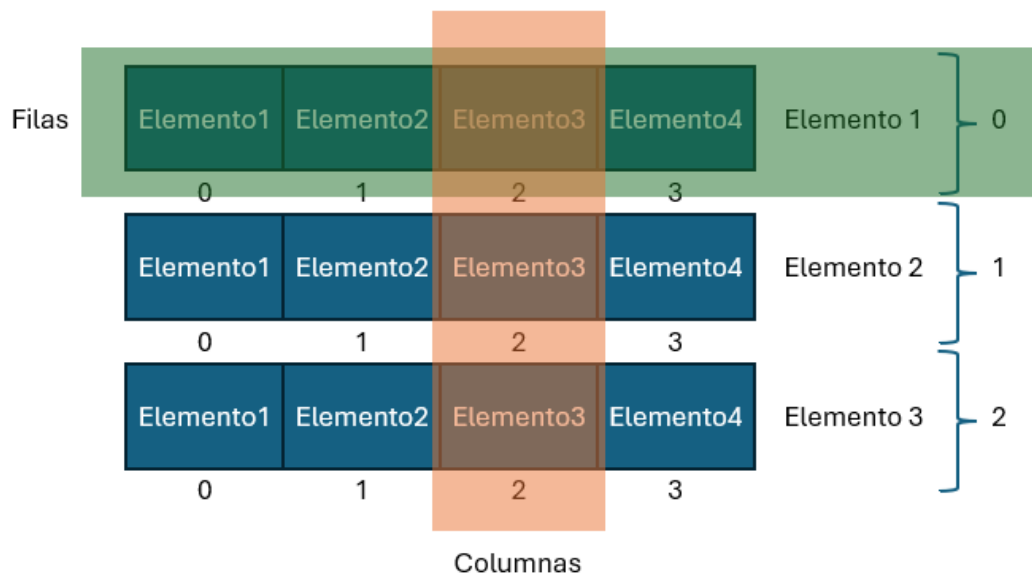


Figura 5. Interpretación de arreglo de dos dimensiones como tabla.

### *¿Como inicializar arreglos de dos dimensiones?*

Existen 2 maneras para inicializar los valores de un arreglo de dos dimensiones: utilizando una lista de valores de 2 dimensiones, o utilizando una lista de valores de una dimensión (Prata, 2004).

#### *Utilizando una lista de valores de 2 dimensiones*

Para ello se indican las listas de valores que componen el array de 2 dimensiones, así como se muestra en el listado 18.

```
int array2dimensiones_a [3][4] = {  
    {1,2,3,4},  
    {5,6,7,8},  
    {9,10,11,12}  
};
```

Listado 18. Ejemplo de inicialización de arreglo de 2 dimensiones.

#### *Utilizando una lista de valores de 1 dimensión*

Los valores de la lista de valores se van asignando a primero a la primera fila elemento por elemento, cuando se terminen los espacios, se empieza a rellenar la siguiente fila elemento por elemento, y así sucesivamente.

```
int array2dimensiones_b [3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
```

Listado 19. Ejemplo de inicialización de arreglo de 2 dimensiones con una lista de inicialización.

Por lo tanto, el array del listado 18 es equivalente al arreglo del listado 19.

Una ventaja de utilizar una lista de valores de una dimensión es que en puedes no definir todos los valores de la lista, y estos se inicializarán a 0. Por ejemplo, en el listado 20, ambos arreglos array2dimensiones\_c y array2dimensiones\_d son equivalentes. Deitel y Deitel (2016) explican esta propiedad en arreglos de una dimensión.

```
int array2dimensiones_c [3][4] = {1,2,3,4,5,6,7};  
  
int array2dimensiones_d [3][4] = {  
    {1,2,3,4},  
    {5,6,7,0},  
    {0,0,0,0}  
};
```

Listado 20. Arreglos equivalentes, demostración de inicialización a 0 para elementos no definidos en la lista de inicialización.

Con esto dicho, puedes inicializar un arreglo de dos dimensiones con todos sus valores a cero de la siguiente manera (listado 21).

```
int arreglo [5][8] = {0};
```

Listado 21. Ejemplo para inicializar todos los elementos de un arreglo 2D a 0.

*¿Como iterar dentro de un arreglo de dos dimensiones?*

Comúnmente cuando se trabaja con arreglos de 2 dimensiones, se deseará leer todo el contenido del array elemento por elemento. El programa del listado 19 muestra una forma de lograrlo utilizando bucles for anidados (Deitel y Deitel, 2016).

```
int array2dimensiones_a [3][4] = {
    {1,2,3,4},
    {5,6,7,8},
    {9,10,11,12}
};

int main()
{
    for (int filas=0; filas<3; filas++){
        for (int columnas=0; columnas<4; columnas++){
            printf("%d \n", array2dimensiones_a[filas][columnas]);
        }
    }
}
```

Listado 19. Ejemplo para iterar a través de todos los elementos de un arreglo bidimensional.

La salida del programa del listado 19 es lo que se muestra en el listado 20. Obsérvese que se imprime cada uno de los elementos del arreglo bidimensional.

```
1
2
3
4
5
6
7
8
9
10
11
12
```

Listado 20. Datos mostrados en la terminal al ejecutar el programa del listado 19.

El programa del listado 19 muestra como leer los elementos del arreglo bidimensional, pero de igual forma utilizando bucles for anidados se puede escribir los elementos que se almacenarán en el arreglo bidimensional.

## Memoria dinámica

### *Función malloc()*

La función malloc() te permite reservar memoria durante la ejecución del programa (Prata, 2004).

Esta función recibe como argumento la cantidad de bytes que se desean reservar, y retorna un puntero a void hacia el primer elemento del bloque reservado. En caso de que no haya memoria disponible, malloc() retorna NULL (Prata, 2004).

Para poder utilizar la función malloc(), es necesario incluir el archivo de cabecera stdlib.h.

Comúnmente se utiliza con la función sizeof(), esta función retorna la cantidad de bytes que ocupa el elemento que se le mande como argumento (Deitel y Deitel, 2016).

En el listado 21 se muestra un ejemplo de uso de malloc(), donde se puede observar lo siguiente:

- Primero se declara el puntero “pointer” que va a recibir el puntero que retorna malloc().
- malloc(3\*sizeof(int)); reserva la memoria necesaria para almacenar 3 datos tipo int.

```
int *pointer;  
pointer = malloc(3*sizeof(int));
```

Listado 21. Ejemplo de uso de la función malloc().

### *Manejando bloques de memoria reservada como arreglos*

Tomando en cuenta que malloc() retorna un puntero al primer elemento del bloque de memoria reservado, entonces se pueden escribir y leer el contenido del bloque de memoria igual como si se tratase de un arreglo (Prata, 2004).

El listado 22 muestra cómo se reserva suficiente memoria para almacenar 5 datos tipo int, y posteriormente se accede a sus elementos con notación de arreglo.

```
int *pointer;
```

```
pointer = malloc(5*sizeof(int));

pointer[0] = 10;
pointer[1] = 9;
pointer[2] = 8;
pointer[3] = 7;
pointer[4] = 6;
```

Listado 22. Ejemplo de manejo de memoria reservada como arreglo.

### *Función free()*

Cuando ya no necesitas un bloque de memoria que hayas reservado, es necesario liberar ese espacio de memoria, no liberarlo provocaría lo que se le conoce como *memory leaks*.

El libro C How to program de Deitel y Deitel (2016) define memory leaks de la siguiente manera: “No liberar memoria reservada cuando ya no se necesita causa que el sistema se le acabe la memoria de manera prematura” (p. 734).

La función free recibe como argumento el puntero que retorna la función malloc. El listado 23 muestra un ejemplo de como utilizar esta función. Primero se crea un puntero que recibirá el puntero que retorna malloc(). Con la función malloc se reserva un espacio de memoria suficiente para almacenar un dato de tipo flotante, y finalmente free() libera ese espacio de memoria reservado.

```
float *pointer;
pointer = malloc(sizeof(float));
free(pointer);
```

Listado 23. Ejemplo de uso de función free().

Siempre cuando se termine la ejecución de un programa se libera toda la memoria reservada (Prata, 2004). Entonces en el programa del listado 24 realmente no es obligatorio usar free(), ya que free() es la última línea en ejecutarse antes de terminar de ejecutarse el programa.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *pointer;
    pointer = malloc(5*sizeof(int));

    for (int i = 0; i < 5; i++){
```

```

        pointer[i] = i;
    }

    for (int i = 0; i < 5; i++){
        printf("%d\n", pointer[i]);
    }

    free(pointer); //No es necesario
}

```

Listado 24. Ejemplo de programa donde al final se libera la memoria reservada utilizando free().

Es importante aclarar que el ejemplo del listado 24 es un programa sencillo y es fácil para el usuario visualizar que la memoria se reserva cuando se llama a malloc() y se libera al término de la ejecución del programa; no obstante, para programas más complejos donde se trabaje con memoria dinámica, bucles, funciones, etc. es imprescindible el uso de free(), ya que evitará que haya *memory leaks*.

## Arreglos de longitud variable

*¿Qué son los arreglos de longitud variable?*

Los VLA (Variable-Length Array) son arreglos los cuales sus dimensiones están especificadas por medio de variables (Prata, 2004).

Es común la confusión de que los VLA consisten en arreglos los cuales su longitud puede variar. La longitud de los VLA una vez definidos siempre conservan su mismo tamaño, “variable” viene de que se utilizan variables para definir las dimensiones del arreglo (Prata, 2004).

El listado 25 muestra la declaración de un arreglo, el cual la cantidad de elementos que va a contener está especificada por la variable tamaño.

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int tamaño = 8;
    int arreglo[tamaño];

    for (int i = 0; i < tamaño; i++){
        arreglo[i] = i;
    }
}

```

Listado 25. Ejemplo de declaración de un VLA.

Es importante indicar, que no se puede declarar e inicializar un VLA al mismo tiempo (Kochan, 2015), por eso en el listado 25 primero se declara y luego se inicializan todos sus valores utilizando un bucle for.

### *VLA como parámetros en funciones*

Una de las grandes ventajas de trabajar con VLAs, es que permiten hacer que puedas diseñar funciones que trabajen con arreglos de cualquier dimensión (Kochan, 2015).

Observemos el caso del programa del listado 26. Se tiene una función `print_data_matrix()` que imprime los datos de un arreglo que reciba como argumento. El problema es que esta función solo recibe arreglos con 4 filas y 3 columnas.

```
#include <stdio.h>

void print_data_matrix (int arg_matrix[4][3]);

int main (void)
{
    int matrix [4][3] = {
        {4,8,6},
        {1,1,7},
        {2,1,8},
        {7,6,1}
    };

    print_data_matrix (matrix);
}

void print_data_matrix (int arg_matrix[4][3]){
    for (int i = 0; i < 4; i++){
        for (int j = 0; j < 3; j++){
            printf("%d\n", arg_matrix[i][j]);
        }
    }
}
```

Listado 26. Programa con función que no usa VLAs.

El listado 27 muestra el programa anterior pero ahora utilizando VLA. Ahora la función “print\_data\_matrix()” puede recibir matrices de cualquier tamaño.

El programa primero llama a la función “print\_data\_matrix()” enviando como argumento un arreglo con 4 filas y 3 columnas, y posteriormente se vuelve llamar a la función “print\_data\_matrix()” pero ahora enviando como argumento un arreglo con 3 filas y 5 columnas.

```
#include <stdio.h>

void print_data_matrix (int filas, int columnas, int arg_matrix[filas][columnas]);

int main (void)
{
    int matrix_a [4][3] = {
        {4,8,6},
        {1,1,7},
        {2,1,8},
        {7,6,1}
    };

    print_data_matrix (4,3, matrix_a);

    printf("\n");

    int matrix_b [3][5] = {
        {1,1,7,2,1},
        {6,4,3,6,0},
        {7,9,3,5,1}
    };

    print_data_matrix (3,5, matrix_b);
}

void print_data_matrix (int filas, int columnas, int arg_matrix[filas][columnas]){
    for (int i = 0; i < filas; i++){
        for (int j = 0; j < columnas; j++){
            printf("%d\n", arg_matrix[i][j]);
        }
    }
}
```

Listado 27. Ejemplo de programa que llama a una función que tiene VLA como parámetro.

Es importante que, en la función, los parámetros que determinan las filas y columnas sean declarados antes que el arreglo (Kochan, 2015). Es decir, que tratar de implementar la función como se muestra en el listado 28 es incorrecto, debería hacerse como se describe en el listado 27.

```
void print_data_matrix (int arg_matrix[filas][columnas], int filas, int columnas);
```

Listado 28. Manera incorrecta de declarar una función que tiene como parámetro un VLA.

### *Memoria dinámica y VLA*

La memoria dinámica y los VLA permiten crear arreglos durante la ejecución del programa (Prata, 2004). Sin embargo, tienen algunas diferencias que se describen en la tabla 1.

**Tabla 1.**

*Diferencias entre memoria dinámica y VLA.*

| <b>Memoria dinámica</b>                                                                                                                         | <b>Variable-Length Array</b>                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Tienes que liberar la memoria usando free()                                                                                                     | Tiene almacenamiento automático.                                                                                   |
| Si creas un arreglo con memoria dinámica dentro de una función, el arreglo seguirá existiendo, aunque la función se haya terminado de ejecutar. | Si creas un VLA dentro de una función, el arreglo deja de existir cuando la función se haya terminado de ejecutar. |
| Como el bloque de memoria este asociado a un puntero se puede acceder al bloque desde cualquier parte del programa.                             | Si creas un VLA dentro de una función, el VLA está limitado al alcance de la función.                              |

Nota. Adaptado de Prata, 2004, p.432.

## **El perceptrón**

### *Elementos básicos del modelo de una neurona*

Haykin (2009) habla sobre los elementos básicos de una neurona artificial. El modelo de la neurona se describe con la figura 6, la cual tiene los siguientes elementos:

1. Un conjunto de sinapsis de señales de entradas asociadas a un peso.
2. Un sumador, que se encarga de sumar todas las señales de entradas ya multiplicadas por su peso asociado.
3. Una función de activación. Limita la amplitud de la salida de la neurona.
4. Un bias, que se encarga de incrementar o disminuir el valor de la entrada total de la función de activación.

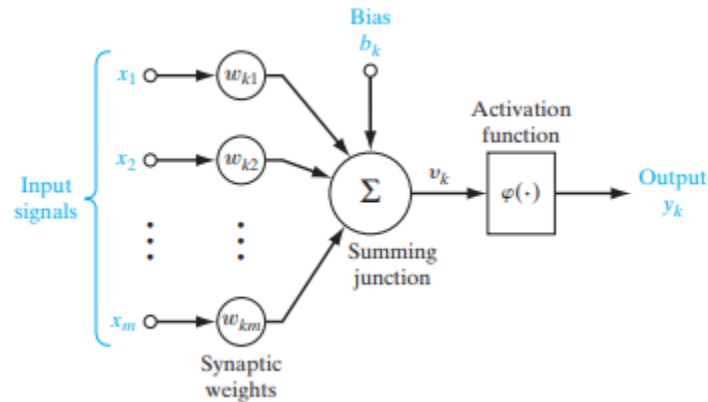


Figura 6. Modelo no lineal de una neurona. Obtenido de Haykin (2009).

#### Modelo matemático de una neurona

Haykin (2009) describe el modelo matemático de una neurona. El diagrama de la figura 5 se puede describir matemáticamente con las siguientes ecuaciones:

$$u_k = \sum_{j=1}^m w_{kj} x_j$$

$u_k$  Representa la suma de la cada una de las entradas multiplicada por sus pesos asociados.

$$y_k = \varphi(u_k + b_k)$$

- $y_k$  Señal de salida de la neurona.
- $u_k + b_k$  El resultado de la combinación lineal + bias.
- $\varphi$  es la función de activación.

#### Función de activación: Umbral

Haykin (2009) describe distintas funciones de activación, entre ellas la función umbral. La función umbral se describe matemáticamente de la siguiente manera:

$$\phi(v) = \begin{cases} 1, & v \geq 0 \\ 0, & v < 0 \end{cases}$$

Y su comportamiento se describe gráficamente en la figura 7.

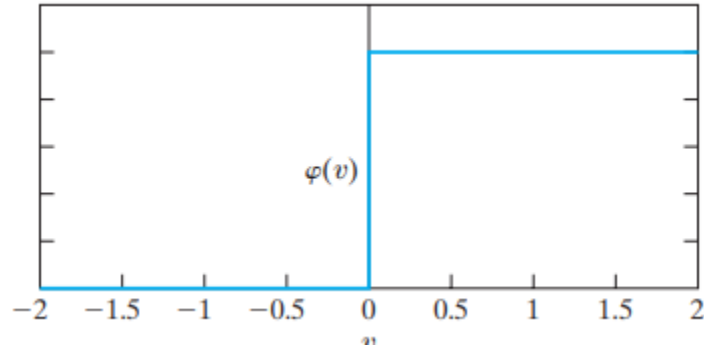


Figura 7. Gráfica de función umbral. Obtenido de Haykin (2009).

#### *Algoritmo: LMS (Least-Mean-Square)*

El algoritmo LMS como un método que por medio de iteraciones actualiza los coeficientes del filtro para minimizar el error cuadrático entre la señal actual y la señal deseada. (Shalini, 2025)

Shalini (2025) describe el flujo del algoritmo LMS, menciona que empieza inicializando los coeficientes del filtro, así como otros parámetros como el learning rate y el número de iteraciones.

Luego de eso, comienza el proceso iterativo:

1. Primero se calcula la salida del filtro, que consiste en la sumatoria de todos los valores de entrada multiplicado por sus respectivos coeficientes.
2. Luego se calcula la señal de error, que consiste en la diferencia entre la señal deseada y la salida actual del filtro.
3. Después se actualizan los coeficientes considerando la siguiente ecuación:

$$w(n + 1) = w(n) + \eta e(n)x(n)$$

La interpretación de la ecuación es que los nuevos pesos  $w(n + 1)$ , consisten en los pesos actuales  $w(n)$  más una actualización dada por  $\eta e(n)x(n)$ , que  $\eta$  es la magnitud del cambio,  $e(n)$  es la señal de error y  $x(n)$  son las entradas que participaron en la iteración.

## Bibliografía

Deitel, P., & Deitel, H. (2016). *C How to Program* (Octava ed.).

Haykin, S. (2009). *Neural networks and learning machines* (Tercera ed.). Pearson Education.

*impl Rust for ESP32.* (s.f.). Abstraction Layers:  
<https://esp32.implrust.com/abstraction-layers.html#abstraction-layers>

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming Language*. Prentice-Hall.

Kochan, S. G. (2015). *Programming in C* (Cuarta ed.). Pearson Education.

Noergaard, T. (2005). *Embedded Systems Architecture*. Elsevier.

Prata, S. (2004). *C Primer Plus* (Quinta ed.). Sams.

Sami, B. (2025). linkedin: [https://www.linkedin.com/posts/bahlous-sami-\\_in-embedded-software-especially-when-working-share-7368277946035228673-9vig/](https://www.linkedin.com/posts/bahlous-sami-_in-embedded-software-especially-when-working-share-7368277946035228673-9vig/)

Shalini. (23 de Julio de 2025). *Least Mean-Squares Algorithm in Neural Networks*.  
geeksforgeeks: <https://www.geeksforgeeks.org/deep-learning/least-mean-squares-algorithm-in-neural-networks/>

Zhu, Z. (16 de junio de 2026). *¿Qué es el firmware en los sistemas embebidos y por qué es importante?* Blue Tech Wave: <https://btw.media/es/what-is-firmware-in-embedded-systems-and-why-is-it-important>